

title: bigmodels-datasets
date: 2025-06-18 23:38:27
tags: llamafactory的./data

一、查看readme_zn.md文件

1. Alpaca格式
- 2.ShareGPT格式详解
- 3.Guanaco格式
- 4.ShareGPT、Alpaca与Guanaco格式的区别区别
- 5.注意事项
- 6.涉及的文件类型（物理存储格式）
 - 6.1 Json
 - 6.2 JSONL（JSON Lines）
 - 6.3 CSV
 - 6.4 Parquet
 - 6.5Arrow
 - 6.6注意事项
- 7.KTO（Knowledge-Enhanced Task-Oriented）数据集简介
- 8.KTO和其他三种格式的对比

二、想微调特定领域的 DeepSeek，数据集究竟要怎么搞？

lmy20250619

一、查看readme_zn.md文件

目前我们支持 **alpaca** 格式和 **sharegpt** 格式的数据集。允许的文件类型包括 json、jsonl、csv、parquet 和 arrow。

1. Alpaca格式

来源：源自Stanford的Alpaca项目，单论指令微调（类似问答对）

典型结构：

```
{  
  "instruction": "任务的描述",  
  "input": "可选输入（可为空）",  
  "output": "期望的输出"  
}
```

例如：

```
{  
  "instruction": "给出以下任务的步骤",  
  "input": "如何泡茶？",  
  "output": "1. 烧水... 2. 预热茶壶..."  
}
```

```
{"instruction": "翻译成英文", "input": "你好", "output": "Hello"}
```

特点:

- 三字段结构: instruction (必选)、input (可选)、output (必选)
- input字段可为空字符串, 表示无需额外输入
- 强调单轮指令跟随任务

2.ShareGPT格式详解

多轮对话数据。

典型结构

```
{
  "conversations": [
    {"from": "human", "value": "用户输入"},
    {"from": "gpt", "value": "AI回复"}
  ]
}
```

例如:

```
[
  {
    "conversations": [
      {
        "from": "human",
        "value": "你好, 你是谁? "
      },
      {
        "from": "gpt",
        "value": "我是一个AI助手, 由XXX训练。"
      },
      {
        "from": "human",
        "value": "你能做什么? "
      },
      {
        "from": "gpt",
        "value": "我可以回答问题、提供建议等。"
      }
    ]
  }
]
```

```
{"conversations": [{"from": "human", "value": "你好"}, {"from": "gpt", "value": "你好!"}]}
```

简化版

```
{
  "id": "sample123",
  "conversations": [
    {"role": "user", "content": "你好！"},
    {"role": "assistant", "content": "你好，有什么可以帮您？"}
  ]
}
```

关键字段

- id: 可选，样本唯一标识符。
- conversations: 必选，列表形式存储对话轮次。每轮对话包含两个字段：
 - from/role: 发言者标识（如 human, user, gpt, assistant 等）。
 - value/content: 对话内容。

3.Guanaco格式

来源：基于HuggingFace的Guanaco项目（QLoRA的低资源微调方案）

典型结构：

```
{
  "text": "<s>[INST] 如何泡茶？ [/INST] 1. 烧水... 2. 预热茶壶...</s>"
}
```

多轮对话结构

```
{
  "text": "<s>[INST] 第一步？ [/INST] 烧水.</s><s>[INST] 然后？ [/INST] 预热茶壶.</s>"
}
```

特点：

- 单字段text包含完整对话标记
- 使用特殊token（如[INST]、）明确划分角色
- 支持多轮对话拼接
- 与HuggingFace Transformer库的tokenizer更深度集成

4.ShareGPT、Alpaca与Guanaco格式的区别区别

特性	ShareGPT格式	Alpaca格式	Guanaco格式
设计目标	多轮对话	单轮指令跟随	指令+多轮混合
角色标识	显式标注 (user/assistant)	无角色区分	通过标记隐式区分 ([INST])
输入输出结构	对话列表	多字段 instruction+output	单字段（text），拼接的文本流
预处理复杂度	需处理对话轮次顺序	直接拼接，需手动拼接	需解析特殊标记，原生支持多轮
适用场景		简单指令任务	复杂对话或QLoRA微调

5. 注意事项

角色名称兼容性:

ShareGPT数据中角色字段可能因来源不同而异(如 human/gpt、user/assistant)，需在预处理时统一。例如，LLaMA Factory可能要求强制转换为 human 和 gpt。

对话完整性: 必须保证对话轮次交替(用户→AI→用户→AI...)，否则训练时会报错。

数据清洗: 需过滤无效对话(如AI先发言、轮次缺失等)。

与其他格式的转换:

ShareGPT→Guanaco: 将多轮对话拼接为 [INST]...[/INST] 形式。

ShareGPT→Alpaca: 通常只取第一轮用户输入作为instruction，第一轮AI回复作为output。

示例: ShareGPT数据预处理脚本，将ShareGPT格式转为Guanaco的单text格式:

```
def sharegpt_to_guanaco(conversations):
    text = ""
    for i, turn in enumerate(conversations):
        if turn["from"] == "human":
            text += f"<s>[INST] {turn['value']} [/INST] "
        else:
            text += f"{turn['value']} </s>"
    return {"text": text}
```

6. 涉及的文件类型 (物理存储格式)

6.1 Json

特点: 标准JSON文件，通常为单个数组包含所有样本。

示例 (alpaca) :

```
[
  {"instruction": "...", "input": "...", "output": "..."},
  {"instruction": "...", "input": "...", "output": "..."}
]
```

6.2 JSONL (JSON Lines)

特点: 每行一个独立JSON样本，适合流式读取。

示例 (sharegpt)

```
{"conversations": [{"from": "human", "value": "..."}]}
```

```
{"conversations": [{"from": "human", "value": "..."}, {"from": "gpt", "value": "..."}]}
```

Alpaca格式 + JSONL文件

```
{"instruction": "翻译", "input": "Hello", "output": "你好"}
```

```
{"instruction": "总结", "input": "长文本...", "output": "摘要..."}
```

6.3 CSV

特点: 表格形式，需与数据集格式字段对应。

示例:

```
instruction,input,output
"翻译成英文","你好","Hello"
"写一首诗","","春眠不觉晓..."
```

6.4 Parquet

特点：列式存储，二进制压缩，适合大规模数据。
结构：需将字段映射为列（如Alpaca的instruction/input/output各为一列）
CSV/JSON转Parquet：可用pandas + pyarrow
示例：ShareGPT格式 + Parquet文件

```
conversations (列)
[{"from":"human", "value":"..."}, {"from":"gpt", "value":"..."}]
[{"from":"human", "value":"..."}]
```

6.5 Arrow

特点：内存高效二进制格式，适合快速加载。
生成方式：通常通过pyarrow库将JSON/CSV转换而来。

6.6 注意事项

字段一致性：无论何种文件类型，内部字段必须符合alpaca或sharegpt的格式规范。
预处理工具：
CSV/JSON转Parquet：可用pandas + pyarrow。
验证格式：通过LLaMA Factory的data_utils.py检查。（不确定）
编码问题：CSV/JSON文件建议保存为UTF-8编码。

文件类型	适用场景	优点	缺点
JSON	小规模数据	可读性强	加载慢，内存占用高
JSONL	中大规模流式处理	支持逐行读取	需手动处理行
CSV	表格友好数据	通用性强	嵌套结构需扁平化
Parquet	超大规模数据	高压缩比，列式查询	需额外工具查看
Arrow	高频加载数据	内存高效，零拷贝	文件体积较大

7.KTO（Knowledge-Enhanced Task-Oriented）数据集简介

KTO 是一种**面向知识增强型任务**的指令微调数据集，主要用于训练大模型（如LLaMA、ChatGLM等）在特定领域的任务表现。它的核心特点是**结合领域知识（Knowledge）与任务指令（Task-Oriented）**，使模型在遵循指令的同时，能有效利用外部知识库或结构化数据来生成更准确的回答。

特点：

特性	说明
知识增强	每条数据可能关联外部知识（如数据库、百科、专业文献等）
任务导向	指令明确指向特定任务（如医疗问答、法律咨询、技术文档生成等）
结构化输入	通常包含 instruction（任务描述）+ knowledge（相关知识片段）
复杂输出	输出可能需要结合知识推理（如多步计算、逻辑判断等）

KTO数据格式示例

(1) 基础格式（类似Alpaca但扩展knowledge字段）

```
{
  "instruction": "基于以下医学知识回答问题。",
  "knowledge": "糖尿病患者的血糖控制目标为空腹血糖4.4-7.0mmol/L。",
  "input": "患者的空腹血糖是8.2mmol/L，是否达标？",
  "output": "不达标，建议调整治疗方案。"
}
```

(2) 多轮对话格式（类似ShareGPT但增加知识引用）

```
{
  "conversations": [
    {
      "from": "human",
      "value": "糖尿病的血糖控制目标是多少？",
      "knowledge": ["医学指南2023版..."]
    },
    {
      "from": "gpt",
      "value": "空腹血糖应控制在4.4-7.0mmol/L。"
    }
  ]
}
```

KTO的典型应用场景

1.垂直领域问答

例如医疗、法律、金融等需要精确知识的领域。
输入: "根据《公司法》第26条，注册资本认缴期限是多久？"
输出: "认缴期限由公司章程约定，但最长不得超过20年。"

2.数据驱动的生成任务

```
{
  "instruction": "根据销售数据生成总结",
  "knowledge": {"Q1销售额": 120万, "Q2销售额": 150万},
  "output": "Q2销售额环比增长25%。"
}
```

3.多模态知识调用

结合文本+图像知识（如医学影像描述）。

8.KTO和其他三种格式的对比

数据集	核心目标	是否需要外部知识	典型任务
Alpaca	通用指令跟随	否	翻译、摘要、简单问答
ShareGPT	多轮开放域对话	否	聊天、角色扮演
Guanaco	低资源高效微调	否	指令压缩、标记优化
KTO	知识增强型任务	是	专业领域问答、数据分析

二、想微调特定领域的 DeepSeek，数据集究竟要怎么搞？
